

Four Tricks at the Edge of Formal Language Theory

15 Aug 2026

Constructions that look like they cheat, and what they actually turn out to be about.

There's a particular feeling you get learning formal language theory and spotting something that shouldn't be allowed. The rules say you can't do X. You find a way to do X. For a moment it feels like you've caught the theory napping.

Usually you haven't. But the interesting part is *why* — and occasionally the thing you almost caught is real, standing slightly to the left of where you were looking. Once below it's a genuine construction; once, a genuine theorem.

Contents

- [Basics for those who were sleeping in class](#)
- [Trick 1: A grammar that can sort numbers](#) — bubble sort as a rewriting system
- [Trick 2: Radix sort in a context-sensitive grammar](#) — beating the quadratic bound
- [Trick 3: The backspace loophole](#) — repairing nested HTML across two formalisms
- [Trick 4: A DFA as a ROM](#) — a finite automaton as storage

Basics for those who were sleeping in class

Four levels, each strictly more expressive than the last. The hierarchy is a story about how much scratch paper you get:

- **Regular.** You can remember which situation you're in, and nothing else.
- **Context-free.** You can remember what you opened, so you can match it when you close it.
- **Context-sensitive.** You get scratch paper, but never more of it than the input you were handed.
- **Recursively enumerable.** You get as much scratch paper as you want.

Regular	⊂	Context-Free	⊂	Context-Sensitive	⊂	Recursively Enumerable
$O(1)$		$O(n)$		$O(n)$		unbounded
states		a stack		a finite tape the size of the input		a tape with no limit at all

That $O(n)$ is a parsing budget, not a generation budget. The linear-bounded automaton is a *recogniser*: hand it a string of length n and it gets a tape of exactly n cells to decide membership, not one cell more. Noncontraction is what makes that enough — since no rule shrinks, no intermediate step in a derivation can be longer than the final string, so checking never needs room beyond the string itself. The tape is finite and sized to the input. It is not "the whole tape" in any Turing sense.

Two things worth having straight:

- **Context-sensitive sits above context-free, not below.** Every context-free rule already satisfies noncontraction — $A \rightarrow \beta$ has one symbol on the left, so a non-empty β is already long enough. Every CFG is a CSG that never uses its context. Strictness comes from $a^n b^n c^n$, which no CFG can produce.
- **Setting $\alpha = \beta = \epsilon$ doesn't collapse the classes.** It makes one *rule* context-free. Restrict every rule that way and you've built a CFG — the subset, not the equality.

Trick 1: A grammar that can sort numbers

A context-free rule takes one non-terminal on the left. A context-sensitive rule takes a whole string, provided the right side is at least as long. Which means you can write a rule that **swaps two adjacent symbols**.

```
 $\Sigma = \{1, 2, 3, 4\}$           no non-terminals, no start symbol

21  $\rightarrow$  12          32  $\rightarrow$  23
31  $\rightarrow$  13          42  $\rightarrow$  24
41  $\rightarrow$  14          43  $\rightarrow$  34
```

One rule per descending adjacent pair — $k(k-1)/2$ for an alphabet of size k . Nothing shrinks, so every rule is legal.

```
4213  $\rightarrow$  2413  $\rightarrow$  2143  $\rightarrow$  1243  $\rightarrow$  1234
      (42)  (41)  (21)  (43)
```

That's bubble sort. Not "analogous to" — it *is* bubble sort, one compare-and-swap per rule application. Apply rules until none apply and you land on the sorted permutation, every time, from any starting point, in any order.

What it turns out to be

- **The language it looks like it defines is regular.** Sorted strings over $\{1, 2, 3, 4\}$ are $1^*2^*3^*4^*$, and the entire grammar is $S \rightarrow 1S \mid T, T \rightarrow 2T \mid U, U \rightarrow 3U \mid V, V \rightarrow 4V \mid \epsilon$. Right-linear, no context anywhere. If the goal was to *describe* sorted strings, the swaps were never needed.
- **It isn't a grammar.** No start symbol, no non-terminals, and it rewrites 1,2,3,4 — which were meant to be *terminals*, and terminals can't be rewritten. That's what makes them terminal.

- **It's a string rewriting system** (a semi-Thue system). Grammars answer "which strings are in the set?" Rewriting systems answer "what does this turn into?" Different objects.
- **The instinct behind it is right, and stronger than the example.** Semi-Thue systems are Turing complete; their word problem is undecidable. Restricting them to noncontracting is exactly what pins them to linear space and gives you the context-sensitive class. This particular rule set is just the tamest fragment available: length-preserving, terminating, confluent.

Hold onto generation-versus-transformation. It comes back.

Trick 2: Radix sort in a context-sensitive grammar

Unlike the other three, this one is LLM-generated. It came out of asking what happens if you take Trick 1's failure seriously instead of just noting it — and the caveats at the end are part of the answer rather than a disclaimer bolted on.

On the name: what's below is strictly counting sort, the per-digit primitive radix sort is built from. With a single-digit alphabet they coincide; run the phases once per digit position and you have genuine LSD radix sort, same rules, more passes.

The quadratic bound isn't sloppiness:

- An adjacent transposition changes the inversion count by exactly one. Feed the rules 44...4 11...1 and that's $n^2/4$ inversions to remove, one per application.
- It survives generalisation. Every rule has a fixed-size left-hand side, so one application shifts total displacement by $O(1)$. Sorting 44...4 11...1 requires $\Theta(n^2)$ displacement.
- **So any local rewriting system that conserves its data symbols is stuck at $\Omega(n^2)$,** however clever the rules.

So stop conserving them. Don't permute the input — destroy it and rebuild from counts. Same move radix sort makes against the comparison-sort bound: decline the premise rather than outsmart it. Comparison sorts are stuck at $n \log n$ because comparisons are all they have; local rewriting is stuck at n^2 because permutation is all it does.

Four symbols means four counters. They can't live in the rule set — a count runs to n — so they live on the tape, in a block that travels with the cursor: $\llbracket c_4 \mid c_3 \mid c_2 \mid c_1 \rrbracket$, each MSB-left.

Phase 1 – sweep right, absorbing input

- | | | | |
|-----|----------------|----------------------------|--|
| (1) | $\llbracket d$ | $\rightarrow d \llbracket$ | absorb the next input symbol |
| (2) | $b d$ | $\rightarrow d b$ | bubble left through the block |
| (3) | $1_d d$ | $\rightarrow 0_d d$ | bit set: clear it, carry continues |
| (4) | $0_d d$ | $\rightarrow 1_d d$ | bit clear: set it, done |
| (5) | $ d$ | $\rightarrow 1_d d$ | overflow: counter gains an MSB [1 $\rightarrow$ 3] |
| (6) | $b d$ | $\rightarrow d b$ | spent traveller bubbles out left |
| (7) | $\llbracket d$ | $\rightarrow \llbracket$ | discarded at the edge |

Phase 2 – sweep left, emitting output

- | | | | |
|-----|--------------|--------------------------------|----------------------------|
| (8) | $\llbracket$ | $\rightarrow \hat{\llbracket}$ | blank enters from the left |
|-----|--------------|--------------------------------|----------------------------|

(9)	$\hat{_} b$	$\rightarrow b \hat{_}$	bubble right through the block
(10)	$\hat{_} 1_d$	$\rightarrow \hat{_}' 0_d$	decrement: clear the low bit
(11)	$\hat{_} 0_d$	$\rightarrow \hat{_} 1_d$	borrow: set and keep going
(12)	$\hat{_}' _]$	$\rightarrow _] d$	emit one literal d at the right edge

The increment falls out for free: bubbling leftward through an MSB-left counter reaches the LSB first, which is the direction carries already travel. Rule (5) is the only one that changes length, and it **grows** — noncontraction only ever forbade shrinking, so the one place a counter needs more room is the one place the formalism is delighted to provide it.

Block width	$O(\log n)$
Advancing one cell	$O(\log n)$
Cells traversed	$2n$
Total	$O(n \log n)$
Space	$n + O(\log n)$ — still linear, still LBA-legal

Caveats:

- **It's a skeleton, not a verified grammar.** Every subscripted d stands for four rules, every b for eight; expanded it runs to several hundred productions. Phase 2 is the shakier half — rules (10) and (11) want the low bit first, but the traveller moves left-to-right through an MSB-first block, so counter 4 needs reversed storage or a return marker.
- **It's a rewriting system, not a grammar** — Trick 1's problem, inherited honestly. Noncontracting semi-Thue is the right home, which is LBA-equivalent, so the label is fair.
- **No optimality claim.** The floor is $\Omega(n)$. Whether the log factor is necessary I don't know; pipelining the counter bits might get $O(n)$ for a constant alphabet, but I haven't worked it through.

Of the four, this is the only one where the loophole instinct cashes out into something that works — and the only one that got there by taking a failure seriously rather than finding a gap in a definition.

Trick 3: The backspace loophole

Noncontraction has a consequence that feels unfair: **a context-sensitive grammar cannot delete.** Not inefficiently — at all, ever. You can mark a symbol dead, since relabelling is length-preserving, but the dead symbol stays in the string forever.

So pick a task where deletion is the point and deciding *what* to delete is beyond anything weaker.

The task: repairing illegally nested HTML

HTML forbids `<p>` inside `<p>`. Browsers silently auto-close the outer one. You want to repair the source: find the nested tags, delete them, emit `<span>` instead — phrasing content, which is legal

where a nested `<p>` isn't.

- **Nesting depth is not context-sensitive.** Tracking depth is exactly what a stack is for. `<p><p></p></p>` is `(( ))`, and balanced brackets are *the* canonical context-free language. Any recursive-descent parser finds every illegal nesting.
- **Tag-name matching is.** A closer must name the same element as its opener, and with custom elements that name is arbitrary-length: `<section-header> ... </section-header>`. A stack can't check it — push `s,e,c...` and popping hands the name back **reversed**. Stacks match a block against its mirror (palindromes, $w w^R$, easy). Matching against a *forward* copy is `{ww}`, which the CFL pumping lemma excludes.

Disclaimer: only if the names are unbounded. Tokenize first — push `section-header` as a single stack symbol — and matching becomes one comparison, reversal problem gone. That works because a PDA's stack alphabet is finite, so it holds for any *bounded* name set, however large. A lexer is finite-state: it can emit the category `IDENT` from a finite alphabet, but not `IDENT(section-header)`, because the payload has nowhere to live. Only genuinely unbounded names force you back to characters. So this is a property of the idealised language rather than anything a real parser meets — which is also why validators keep symbol tables, the implementation conceding it needs storage a stack alphabet can't give it.

- **So the boundary runs through "just parsing HTML."** Fixed vocabulary: the name fits in the stack alphabet, context-free. Open vocabulary: above it.

The grammar

Non-terminals `S, Doc, El, N (tag-name block), P ("you are inside an element")`

Terminals `< > / t (text) \b s,p,a,n`

Structure

- (1) `S → Doc`
- (2) `Doc → El Doc | t Doc | ε`
- (3) `El → < N > P Doc < / N >`

Annotation — fires only when preceded by `P`, i.e. at depth ≥ 2

- (4) `P < p > → P < p > \b\b\b < s p a n >` `[ 4 → 13 ]`
- (5) `P < / p > → P < / p > \b\b\b < / s p a n >` `[ 5 → 16 ]`

Migration

- (6) `P t → t P`
- (7) `\b t → t \b`

Rules (4) and (5) have the shape a context-sensitive rule is defined by — $\alpha A \beta \rightarrow \alpha \gamma \beta$, rewrite `A` only when it sits between α and β . Here: rewrite the tag, but only when a `P` sits to its left. A top-level `<p>` has no `P` before it and nothing fires. Both rules **grow**, so noncontraction is satisfied comfortably. The formalism is happy to let you *add* the replacement; it's the removal it won't allow.

What rule (3) doesn't do: writing `< N > ... < / N >` looks like it forces the names to match. It doesn't — the two Ns derive independently, so nothing stops `<section>...</header>`. Enforcing agreement is the `{ww}` construction (generate name characters in matched pairs, migrate with $N \times \rightarrow x \times N$), and that's where the context-sensitivity actually lives. Rule (3) is quietly standing in for it.

```
<p>Outer<p>Inner</p></p>
→ <p>Outer<p>\b\b\b<span>Inner</p>\b\b\b</span></p>
```

Every original character still present. Nothing deleted. The document has grown, and the instructions for shrinking it sit inline.

Stage 2 — the CFG does the deleting

The strings that type characters and erase them all are the Dyck language wearing a disguise:

```
T → c T \b T
T → ε
```

`c` opens, `\b` closes, a well-formed erasure is a well-formed bracketing.

```
<p>Outer<p>\b\b\b<span>Inner</p>\b\b\b</span></p>
→ <p>Outer<span>Inner</span></p>
```

The CSG decides what to delete, because only a CSG can make that call. The CFG performs the deletion, because only something allowed to shrink can perform it. Neither can do the other's job, and each is blocked from the other by a property of its own definition.

What it turns out to be

- **Grammars generate; they don't transform.** A grammar defines a set of strings — it doesn't take one in and hand a different one back. $T \rightarrow \epsilon$ isn't deleting anything, it just never emitted it. The Dyck grammar describes *which* strings erase; a pushdown automaton recognising it is what actually erases, as a side effect of stack discipline.
- **So the crisp line above is slightly too neat** — it's a generator followed by a transducer. But the architecture survives, and the architecture is the point.

The part that's actually real

Context-sensitive languages are closed under union, intersection, concatenation, Kleene star, reversal, and even complementation (the hard one, settled in 1988). They're closed under λ -free homomorphisms too. They are **not** closed under homomorphism in general, and the single operation that breaks it is **erasure**.

Which is exactly what stage two does.

So this pipeline isn't standing near the escape hatch — structurally, it *is* the escape hatch. A context-sensitive generator feeding an erasing consumer is precisely how a language leaves the class that produced it. The mechanism is padding: stage one emits a long string encoding something, stage two erases the padding, and what survives was never something stage one could have produced directly.

The HTML instance doesn't exercise it, for a reassuringly boring reason — what survives is well-formed HTML, no harder than what went in. Swap the payload for a padded computation history and the same two stages, unchanged, hand you a recursively enumerable language.

The reframing worth keeping: a symbol meaning "undo the previous symbol" changes what kind of question you're asking. `\b` doesn't describe content, it describes an *edit to content* — the string has stopped being a static object you check for membership and become a sequence of instructions you execute. That's language membership versus execution semantics, and it's the same shift that separates a grammar from a transducer.

Trick 4: A DFA as a ROM

Take any fixed binary string w of length n . Build a DFA as a chain of $n+1$ states — one per "how many characters have matched" — with an accepting state at the end and a dead state absorbing everything wrong.

```
q0 --w[0]--> q1 --w[1]--> q2 --...--> qn (accept)
```

As a grammar it's right-linear, one production per character. For $w = 1011$ that's the whole thing:

```
S → 1 A      A → 0 B      B → 1 C      C → 1
```

The rules *contain* the payload, one bit each, in order. The grammar isn't describing the string structurally — it's storing it. Fixed bits, addressed lookup: a ROM, and a nice bridge from automata theory to digital logic.

What it turns out to be

- **$\{w\}$ is always regular**, for any string at all. So is any finite set — build a big enough chain or tree and enumerate.
 - **The pumping lemma was never making a claim here.** It constrains unboundedly large *families* sharing one structural pattern across every length. That's what makes $a^n b^n$ non-regular. It says nothing about a single fixed-length string picked in advance, however long.
 - **The state count is decided at design time.** The automaton has no obligation to generalise to anything it wasn't built for, so it never meets the constraint.
-

Original conversation: claude.ai/share/67dbb2a6-441c-4c87-b34a-5df0351630ab

What do you think?

0 Responses



Upvote



Funny



Love



Surprised



Angry



Sad

0 Comments

1 Login ▼



Start the discussion...

LOG IN WITH

OR SIGN UP WITH DISQUS ?



Name



• Share

Best

Newest

Oldest

All articles

[Four Tricks at the Edge of Formal Language Theory](#)

15 Aug 2026

[Spring-Loaded Dinosaurs: What Penguins, Kiwis, and Armored Core Get Right \(and Wrong\) About Legs](#)

28 Feb 2026

[Passive Velocity Sorting: A Geometric Maxwell's Demon?](#)

14 Dec 2025

[AC Gravity and Gravitational Opacity](#)

05 Dec 2025

[Theorem: Equivalence of \$-\operatorname{arccoth}\$ and \$\tan\$ Functions](#)

21 Nov 2025

[The Photonic Transformer Hypothesis: Rethinking Photosynthesis and Methane Consumption in Plants](#)

19 Nov 2025

[The Scalable Quasi-Perpetual Photonic Machine](#)

17 Nov 2025

[How to Make a Ball Orbit Itself](#)

08 Nov 2025

[The Boron-Nitrogen Catastrophe: A Critical Gap in Beta Decay Verification](#)

28 Oct 2025

[X-Ray Data Pipeline: Ultra-High-Speed Communication Through Limestone Tubes](#)

26 Oct 2025

[The Universal Approximation Theorem Is Right. You're Using It Wrong.](#)

19 Oct 2025

[The Power Law Illusion: A Measurement Artifact Hypothesis](#)

11 Oct 2025

[Fractional Gamma Function via Fractional Derivatives](#)

18 Sep 2025

[On the cruel irony of the P-NP problem](#)

18 Dec 2023

[The Journey from India to Germany: A Guide for IT Professionals](#)

20 Jun 2023

[How does AutoGPT work under the hood?](#)

03 May 2023

[Unit Test Recorder - Automatically generate unit tests as you use your application](#)

25 Jun 2020

[How to migrate from vanilla Kubernetes to Istio service mesh?](#)

14 Oct 2019